“ORDERING / BILLING SYSTEM”

FA2 - BSIT1 - 01

Members:
Amion, Angela
Bael, Kyle
Quezon, Rochen
Garcia, Joren
Abaton, Jerald

Introduction

This Simple Ordering and Billing System offers a streamlined, intuitive interface designed specifically for “Hab-hub”. Developed with Python and Tkinter, it facilitates efficient order management with a clean, easy-to-use design that lets customers securely register, log in, and place orders. Real-time updates on order details ensure a smooth, transparent experience by clearly displaying costs and wait times, ultimately enhancing service and minimizing customer wait periods.

Key functions of the system include secure user registration and login, allowing access to personalized ordering features. The menu is visually engaging, with item images and prices to facilitate easy selection. As customers add or remove items, the system dynamically calculates and displays the total cost and estimated wait time. After placing an order, users can choose between "Dine In" or "Take Out" options and track order status, adding transparency to the process and improving the overall dining experience.

Algorithm

Step 1: Initialize Main Window

Create the main application window using tk.Tk().

Set the title and dimensions of the window.

Step 2: Load Images

Define a function load_image(image_path, scale) to load and scale images for menu items and banners.

Load individual images for menu items (e.g., chicken, BBQ, rice, soda) and banner images.

Step 3: Create Banner Frame

Create a Frame for the banner with a maroon background.

Add a tagline image and a main banner image on the right and left sides of the banner frame.

Step 4: Define Menu Items

Define a dictionary menu_items to hold each menu item with:

price, wait_time, and image data.

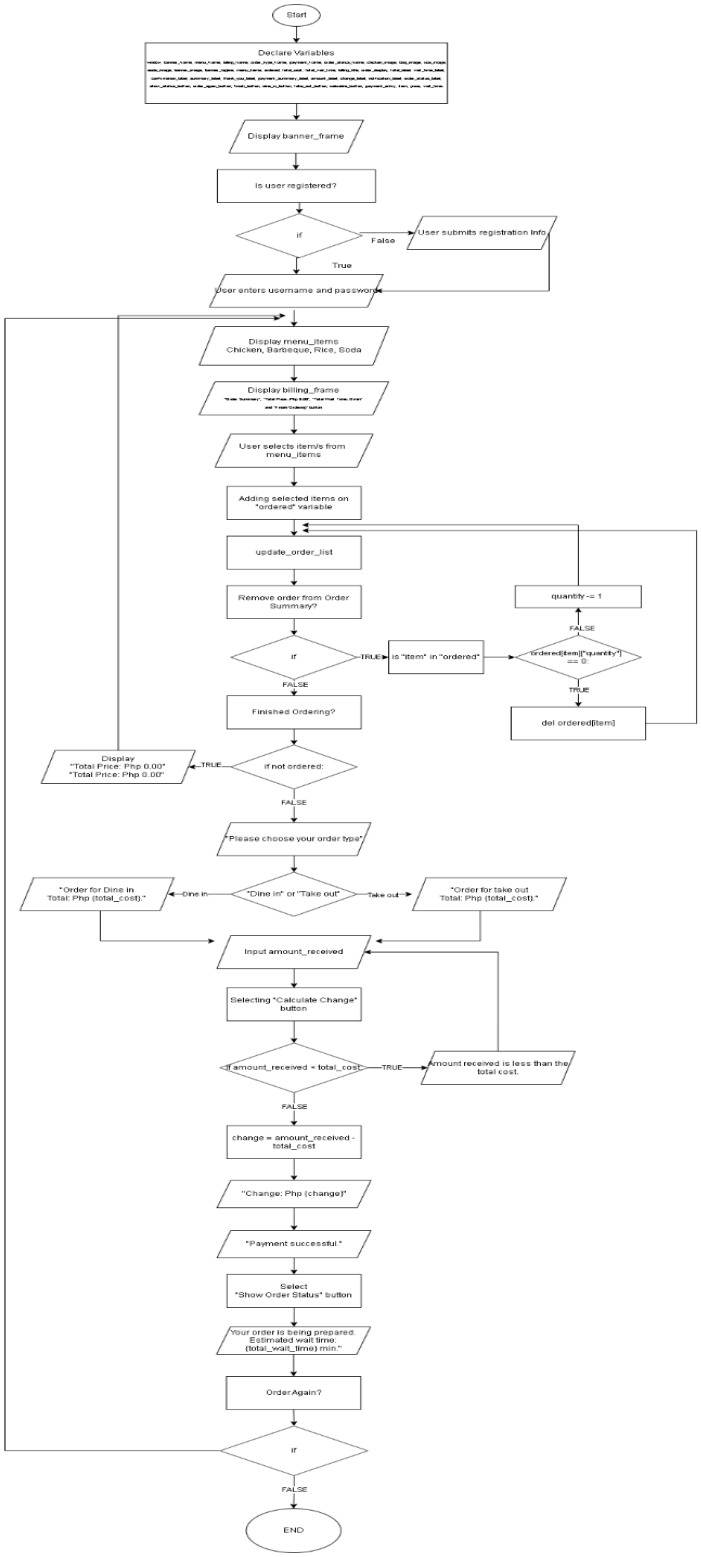
Initialize storage variables:ordered (for tracking orders), total_cost and total_wait_time.

Step 5: Define Functions

Step 6: Create Frames

Step 7: Run the Application

Flowchart:



Text

```
import tkinter as tk
from tkinter import messagebox

# MAIN WINDOW
window = tk.Tk()
window.title("Simple Ordering System")
window.geometry("1000x800")

def load_image(image_path, scale=2):
    image = tk.PhotoImage(file=image_path).subsample(scale, scale)
    return image

# Load images for menu items and banner
chicken_image = load_image("chicken.png")
bbq_image = load_image("bbq.png")
rice_image = load_image("rice.png")
soda_image = load_image("sodda.png")
banner_image = load_image("habhub.png", scale=4)
banner_tagline = load_image("habhub2.png", scale=7)

# Create a banner with a maroon background and an image
banner_frame = tk.Frame(window, bg="maroon", width=1000, height=100)
banner_frame.pack(side=tk.TOP, fill=tk.X)
banner_label_tagline = tk.Label(banner_frame, image=banner_tagline,
bg="maroon")
banner_label_tagline.pack(side=tk.RIGHT, padx=100, pady=10)
banner_label_image = tk.Label(banner_frame, image=banner_image,
bg="maroon")
banner_label_image.pack(side=tk.LEFT, padx=100, pady=10)

# Define menu items with pricing and wait times
menu_items = {
    "Pizza": {"price": 30, "wait_time": 15, "image": chicken_image},
    "Burger": {"price": 50, "wait_time": 10, "image": bbq_image},
    "Fries": {"price": 30, "wait_time": 5, "image": rice_image},
    "Soda": {"price": 20, "wait_time": 2, "image": soda_image}
```

```
}

# Storage for orders
ordered = {}
total_cost = 0.0
total_wait_time = 0
users = {}

# Function to switch to the registration screen
def show_registration():
    menu_frame.pack_forget()
    billing_frame.pack_forget()
    login_frame.pack_forget()
    registration_frame.pack(pady=20)

# Function to register new users
def register_user():
    menu_frame.pack_forget()
    billing_frame.pack_forget()
    username = reg_username_entry.get()
    password = reg_password_entry.get()
    if username in users:
        registration_message_label.config(text="Username already exists!",
fg="red")
    elif not username or not password:
        registration_message_label.config(text="Please enter both username
and password.", fg="red")
    else:
        users[username] = password
        registration_message_label.config(text="Registration successful!",
fg="green")
        show_login()

# Function to log in
def login_user():
    menu_frame.pack_forget()
```

```
    billing_frame.pack_forget()
    username = login_username_entry.get()
    password = login_password_entry.get()
    if users.get(username) == password:
        login_message_label.config(text=f"Welcome, {username}!",
fg="green")
        show_main_app()
    else:
        login_message_label.config(text="Invalid username or password.",
fg="red")

# Function to switch to the registration screen
def show_registration():
    login_frame.pack_forget()
    registration_frame.pack(pady=20)
    menu_frame.pack_forget()
    billing_frame.pack_forget()

# Function to switch to the login screen
def show_login():
    registration_frame.pack_forget()
    login_frame.pack(pady=20)
    menu_frame.pack_forget()
    billing_frame.pack_forget()

# Function to show the main application after login
def show_main_app():
    login_frame.pack_forget()
    registration_frame.pack_forget()
    menu_frame.pack(side=tk.LEFT, padx=20, pady=20)
    billing_frame.pack(side=tk.RIGHT, padx=20, pady=20)

def place_order(item, price, wait_time):
    global total_cost, total_wait_time
    if item in ordered:
        ordered[item]["quantity"] += 1
```

```
    else:
        ordered[item] = {"price": price, "wait_time": wait_time, "quantity": 1}

    total_cost += price
    total_wait_time += wait_time
    update_order_list()

def remove_order(item):
    global total_cost, total_wait_time
    if item in ordered:
        ordered[item]["quantity"] -= 1
        total_cost -= ordered[item]["price"]
        total_wait_time -= ordered[item]["wait_time"]

        if ordered[item]["quantity"] == 0:
            del ordered[item]

        update_order_list()

def update_order_list():
    for widget in order_display.winfo_children():
        widget.destroy()

    for item, details in ordered.items():
        quantity = details["quantity"]
        price = details["price"]
        wait_time = details["wait_time"]

        item_label = tk.Label(order_display, text=f"{quantity}x {item} -
Php{price * quantity} ({wait_time * quantity} min)", font=("Arial", 14))
        item_label.pack()

    total_label.config(text=f"Total Price: Php{total_cost:.2f}")
    wait_time_label.config(text=f"Total Wait Time: {total_wait_time} min")

def show_order_type_selection():
```

```
    billing_frame.pack_forget()  # Hide the billing frame
    menu_frame.pack_forget()      # Hide the menu frame
    order_type_frame.pack(pady=20)  # Show order type selection frame

def select_dine_in():
    show_final_summary("Dine In")

def select_take_out():
    show_final_summary("Take Out")

def show_final_summary(order_type):
    order_type_frame.pack_forget()  # Hide order type selection frame
    billing_frame.pack(pady=20)  # Show billing frame centered

    # Display summary with total cost and wait time
    confirmation_label.config(text=f"Order for {order_type}.")
    summary_label.config(text=f"Total: Php {total_cost:.2f}, Wait Time:
{total_wait_time} min.")
    thank_you_label.config(text="Thank you for your order!")

    # Button to show order status
    show_status_button.pack(pady=10)

def show_order_status():
    billing_frame.pack_forget()  # Hide billing frame
    order_status_frame.pack(pady=20)  # Show order status frame

    # Display the order status information
    order_status_label.config(text=f"Your order is being
prepared.\nEstimated wait time: {total_wait_time} min.")

    # Show the order again button
    order_again_button_status.pack(pady=10)

def finish_order():
    if not ordered:
```

```
        total_label.config(text="Total Price: Php 0.00")
        wait_time_label.config(text="   Total Wait Time: 0 min")
        confirmation_label.config(text="")
        show_status_button.pack_forget()  # Hide the order status button if
there's no order
    else:
        confirmation_label.config(text="Please choose your order type:")
        thank_you_label.config(text="")
        order_again_button.pack_forget()
        finish_button.pack_forget()
        show_order_type_selection()  # Show the order type selection page


def reset_order():
    global total_cost, total_wait_time, ordered
    ordered = {}
    total_cost = 0.0
    total_wait_time = 0
    update_order_list()

    confirmation_label.config(text="")
    summary_label.config(text="")
    thank_you_label.config(text="")
    order_status_label.config(text="")

    order_again_button.pack_forget()
    finish_button.pack(pady=20)
    menu_frame.pack(side=tk.LEFT)
    billing_frame.pack_forget()
    billing_frame.pack(side=tk.RIGHT, padx=20, pady=20)
    order_status_frame.pack_forget()

def reset_order():
    global total_cost, total_wait_time, ordered
    ordered = {}
    total_cost = 0.0
```

```
    total_wait_time = 0
    update_order_list()

    confirmation_label.config(text="")
    summary_label.config(text="")
    thank_you_label.config(text="")
    order_status_label.config(text="")

    # Hide the show status button
    show_status_button.pack_forget()

    order_again_button.pack_forget()
    finish_button.pack(pady=20)
    menu_frame.pack(side=tk.LEFT)
    billing_frame.pack_forget()
    billing_frame.pack(side=tk.RIGHT, padx=20, pady=20)
    order_status_frame.pack_forget()


# Frames
# Login Frame
login_frame = tk.Frame(window)
login_frame.pack(pady=100)

# Add a label in the login frame to display login messages
login_message_label = tk.Label(login_frame, text="", font=("Arial", 12),
fg="red")
login_message_label.pack(pady=5)

login_label = tk.Label(login_frame, text="Login", font=("Arial", 18))
login_label.pack(pady=10)

login_username_label = tk.Label(login_frame, text="Username:",
font=("Arial", 14))
login_username_label.pack(pady=5)
login_username_entry = tk.Entry(login_frame, font=("Arial", 14))
```

```
login_username_entry.pack(pady=5)

login_password_label = tk.Label(login_frame, text="Password:",
font=("Arial", 14))
login_password_label.pack(pady=5)
login_password_entry = tk.Entry(login_frame, font=("Arial", 14), show="*")
login_password_entry.pack(pady=5)

login_button = tk.Button(login_frame, text="Login", font=("Arial", 12),
command=login_user)
login_button.pack(pady=10)

register_switch_button = tk.Button(login_frame, text="Register",
font=("Arial", 10), command=show_registration)
register_switch_button.pack(pady=5)

# Registration Frame
registration_frame = tk.Frame(window)

# Add this label to the registration frame to show messages
registration_message_label = tk.Label(registration_frame, text="",
font=("Arial", 12), fg="red")
registration_message_label.pack(pady=5)
registration_label = tk.Label(registration_frame, text="Register",
font=("Arial", 18))
registration_label.pack(pady=10)

reg_username_label = tk.Label(registration_frame, text="Username:",
font=("Arial", 14))
reg_username_label.pack(pady=5)
reg_username_entry = tk.Entry(registration_frame, font=("Arial", 14))
reg_username_entry.pack(pady=5)

reg_password_label = tk.Label(registration_frame, text="Password:",
font=("Arial", 14))
reg_password_label.pack(pady=5)
```

```
reg_password_entry = tk.Entry(registration_frame, font=("Arial", 14),
show="*")
reg_password_entry.pack(pady=5)

menu_frame = tk.Frame(window)
menu_frame.pack(side=tk.LEFT, padx=20, pady=20)

billing_frame = tk.Frame(window, width=400, height=600)
billing_frame.pack(side=tk.RIGHT, padx=20, pady=20)  # Start with
billing_frame on the right

# Order summary labels and buttons
billing_title = tk.Label(billing_frame, text="Order Summary", font=("Arial",
16))
billing_title.pack(pady=10, anchor='center')
order_display = tk.Frame(billing_frame)
order_display.pack(pady=20)

total_label = tk.Label(billing_frame, text="Total Price: Php 0.00",
font=("Arial", 12))
total_label.pack(pady=10)

wait_time_label = tk.Label(billing_frame, text="Total Wait Time: 0 min",
font=("Arial", 12))
wait_time_label.pack(pady=10)

confirmation_label = tk.Label(billing_frame, text="", font=("Arial", 12))
confirmation_label.pack(pady=10)

summary_label = tk.Label(billing_frame, text="", font=("Arial", 12))
summary_label.pack(pady=10)

thank_you_label = tk.Label(billing_frame, text="", font=("Arial", 12))
thank_you_label.pack(pady=10)

# Button to show order status
```

```
register_button = tk.Button(registration_frame, text="Register",
font=("Arial", 12), command=register_user)
register_button.pack(pady=10)

login_switch_button = tk.Button(registration_frame, text="Back to Login",
font=("Arial", 10), command=show_login)
login_switch_button.pack(pady=5)

show_status_button = tk.Button(billing_frame, text="Show Order Status",
font=("Arial", 12), width=20, height=2, command=show_order_status)
show_status_button.pack_forget()  # Hide until the order is finished

order_again_button = tk.Button(billing_frame, text="Order Again",
font=("Arial", 12), width=20, height=2, command=reset_order)
order_again_button.pack_forget()  # Initially hidden

finish_button = tk.Button(billing_frame, text="Finish Ordering",
font=("Arial", 12), width=20, height=2, command=finish_order)
finish_button.pack(pady=20)

# Order Type Frame
order_type_frame = tk.Frame(window)
order_type_label = tk.Label(order_type_frame, text="Select Order Type",
font=("Arial", 18))
order_type_label.pack(pady=10)

dine_in_button = tk.Button(order_type_frame, text="Dine In", font=("Arial",
14), width=15, height=2, command=select_dine_in)
dine_in_button.pack(pady=10)

take_out_button = tk.Button(order_type_frame, text="Take Out",
font=("Arial", 14), width=15, height=2, command=select_take_out)
take_out_button.pack(pady=10)

# Order Status Frame
order_status_frame = tk.Frame(window)
```

```
order_status_label = tk.Label(order_status_frame, text="", font=("Arial",
16))
order_status_label.pack(pady=20)

# Add "Order Again" button to the order status frame
order_again_button_status = tk.Button(order_status_frame, text="Order
Again", font=("Arial", 12), width=20, height=2, command=reset_order)
order_again_button_status.pack(pady=10)  # Place the button in the frame

# Create buttons for menu items
button_font = ("Arial", 10)
row = 0
col = 0

for item, details in menu_items.items():
    btn_frame = tk.Frame(menu_frame)
    btn = tk.Button(btn_frame, text=f"{item}\nPhp{details['price']}",
font=button_font, width=10, height=2,
                    command=lambda item=item, price=details['price'],
wait_time=details['wait_time']: place_order(item, price, wait_time),
                    relief="solid", borderwidth=2)
    btn.pack()

    img_label = tk.Label(btn_frame, image=details['image'])

    img_label.pack()

    btn_frame.grid(row=row, column=col, padx=10, pady=10)

    col += 1
    if col > 1:
        col = 0
        row += 1


window.mainloop()
```